

PHY407 Formal Lab Report 4

Fall 2024

Background

Gauss-Seidel for Elliptic PDEs To obtain the solution of a field $\phi(x, y)$ that satisfies Laplace's equation

$$\nabla^2 \phi = \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0, \quad (1)$$

subject to boundary conditions, overrelaxation is written [see Newman's Equation (9.17)]:

$$\phi(x, y) \leftarrow \frac{1+\omega}{4} [\phi(x+a, y) + \phi(x-a, y) + \phi(x, y+a) + \phi(x, y-a)] - \omega \phi(x, y), \quad (2)$$

where the left arrow indicates replacement of the left hand side with the right, and ω is a relaxation parameter. For these methods, you can either pick a fixed number of iterations or can choose a level of convergence for the solution. In this lab, we will be using the latter approach.

Flux-conservative PDE problems A large class of time-dependent PDEs can be written in flux-conservative form. In one space dimension, this is given as

$$\frac{\partial \vec{u}}{\partial t} = - \frac{\partial \vec{F}(\vec{u})}{\partial x}, \quad (3)$$

where, in general, $\vec{u}$ and $\vec{F}$ are vectors consisting of a set of multiple fields. The variable we are solving for is $\vec{u}$, while $\vec{F}$ is a given function that can depend on $\vec{u}$ and on spatial derivatives of $\vec{u}$.

The simplest scheme to discretize this system is the forward-time centred-space (FTCS) scheme, in which one uses a forward difference for the time derivative, and a centred difference for the spatial derivative. If, for example, the problem is one dimensional and $\vec{u}$ and $\vec{F}$ have just one component field, then

$$\left. \frac{\partial u}{\partial t} \right|_j^n \approx \frac{1}{\Delta t} (u_j^{n+1} - u_j^n), \quad \left. \frac{\partial F}{\partial x} \right|_j^n \approx \frac{1}{2\Delta x} (F_{j+1}^n - F_{j-1}^n), \quad (4)$$

where superscripts like n refer to the time step index and subscripts like j refer to the spatial index (see Figure 1).

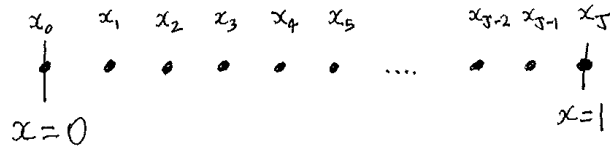


Figure 1: Notations for spatial grid.

Substituting these approximations into eqn. (3) and rearranging, it is easy to show that

$$u_j^{n+1} = u_j^n - \frac{\Delta t}{2\Delta x} (F_{j+1}^n - F_{j-1}^n). \quad (5)$$

Note, as you will notice in this lab, this very simple discretization is unstable for typical wave equations.

Simple electrostatics In empty space, the electrostatic potential $V(x, y)$ satisfies Laplace's equation $\nabla^2 V = 0$. The electric field is $\vec{E} = -\nabla V$ and the electrostatic force felt by a particle with charge q is given by $q\vec{E}$. A suitable equation of motion for the particle is $\ddot{\mathbf{x}} = q\vec{E}/m$, where m is the mass of the charged particle. We can use this equation of motion a bit more broadly, if the electric fields are slowly varying in time and the particle isn't moving too fast.

In this lab, you will be calculating an electric field with strong spatial variations and allowing it to oscillate in time. This kind of configuration can be used to confine charged particles, based on the idea that charged particles in an oscillating electric field tend to be attracted to regions where the electric field strength is weak.

Inviscid Burger's Equation This is a classical nonlinear wave equation which, in one spatial dimension x , takes the form

$$\frac{\partial u}{\partial t} + \epsilon u \frac{\partial u}{\partial x} = 0 \quad (6)$$

or, equivalently,

$$\frac{\partial u}{\partial t} + \epsilon \frac{\partial}{\partial x} \left(\frac{u^2}{2} \right) = 0 \quad (7)$$

where $u(x, t)$ is the speed of the wave. In the generalized form, ϵ can be any arbitrary function of u . In this lab, though, we will take ϵ to be a constant. In this case, the equation is just like a wave propagation equation in which the phase speed of the wave v is replaced by the actual speed of the wave u . This nonlinearity allows the wave to steepen. It is one way to model water waves approaching a beach, for example.

Shallow Water Equations Now for another way to model water waves. The “shallow water equations” are a simplification of the full Navier-Stokes equations (which generally describe the motion of fluid flow), appropriate for describing the motion in shallow layers of fluid under the influence of gravity. By “shallow” we mean that the depth of the fluid is small compared to the relevant horizontal length scales of the fluid. Thus, in some cases, the ocean can be considered a shallow fluid, as long as we are interested in phenomena whose horizontal length scales are much larger than the depth of the ocean (on average, around 4 km). In particular, it is common to use the shallow water equations for modelling the tides, and for simulating tsunamis in the ocean.

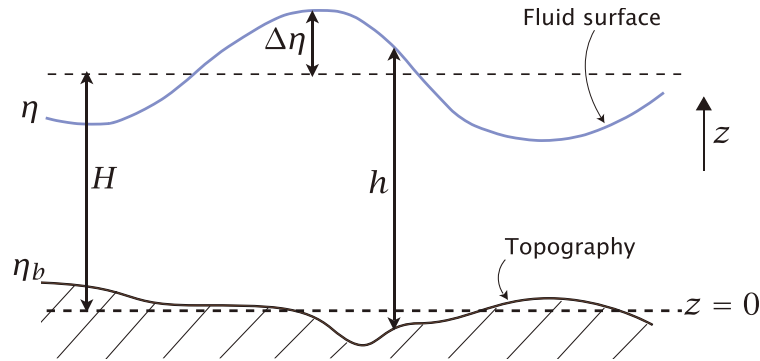


Figure 2: Variable definitions for the shallow water system. This diagram shows a cross-section on the (x, z) plane. The upper dashed line represents the equilibrium surface, the lower dashed line the $z = 0$ origin, H is the average (in time and space) water column height, the top solid line represents the free surface of the fluid at $z = \eta(x, t)$, and the bottom solid line represents the fixed bottom topography at $z = \eta_b(x)$. From Vallis (2017, fig. 3.1).

For simplicity, we will restrict ourselves to the 1D version of the shallow-water equations:

$$\begin{aligned}\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} &= -g \frac{\partial \eta}{\partial x} \\ \frac{\partial \eta}{\partial t} + \frac{\partial (uh)}{\partial x} &= 0\end{aligned}\tag{8}$$

where u is the fluid velocity in the x direction, $h = \eta - \eta_b$ is the water column height, η the altitude of the free surface, η_b is the altitude of the bottom topography (which is typically a known function), and g is the acceleration due to gravity. (See also Fig. 2 for notations.) In this lab, we will also restrict ourselves to constant η_{ab} , meaning a flat bottom. Note that all the variables are functions of x and t , but not z (i.e. every column of fluid moves together), as a consequence of the assumptions that lead to the shallow water equations.

These equations support wave solutions in which the waves are non-dispersive; their phase and group speeds over a flat bottom are equal to $\sqrt{gH}$, where H is the water column height at rest.

Stream Plots This type of plot is used to illustrate a vector field $\vec{F}$. At each location, $\vec{F}(\vec{x})$ (e.g. electric field) is tangential to the local streamline (e.g. electric field line). Matplotlib's `streamplot` function (reference page https://matplotlib.org/3.2.1/api/_as_gen/matplotlib.pyplot.streamplot.html) can colour-code the lines with the scalar of your choice. Adapted from this page is the example below, for electric field lines due to the presence of two point charges $\pm 4\pi\epsilon_0$, separated by a distance $d = 2$.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2, 2, 100) # does not include zero
y = np.linspace(-1, 1, 50)
X, Y = np.meshgrid(x, y)
R1 = ((X-1)**2 + Y**2)**.5 # 1st charge located at x=+1, y=0
R2 = ((X+1)**2 + Y**2)**.5 # 2nd charge located at x=-1, y=0

V = 1./R1 - 1./R2 # two equal-and-opposite charges
Ey, Ex = np.gradient(-V, y, x) # careful about order

fig = plt.figure(figsize=(6, 3))
strm = plt.streamplot(X, Y, Ex, Ey, color=V, linewidth=2, cmap='autumn')
cbar = fig.colorbar(strm.lines)
cbar.set_label('Potential $V$')
plt.title('Electric field lines')
plt.xlabel('$x$')
plt.ylabel('$y$')
plt.axis('equal')
plt.tight_layout()
plt.show()
```

Questions

1. Electrostatics and Laplace's equation

- (a) Consider the simple model of an electronic capacitor sketched in Fig. 3, consisting of two flat metal plates enclosed in a square metal box.

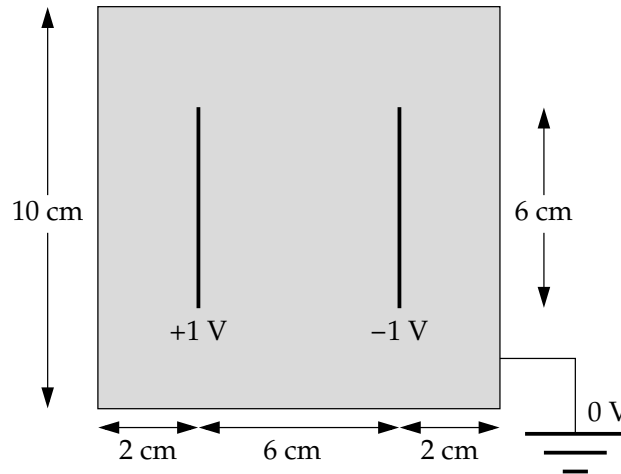


Figure 3: Capacitor for Question 1 (from Newman, p. 417).

For simplicity let us model the system in 2D. Using the Gauss-Seidel method without over-relaxation, write a program to calculate the electrostatic potential at each grid point in the box, with the following conditions:

- Grid of 100×100 points
- Walls of the box at 0 voltage
- The two plates (of negligible thickness) at voltages ± 1 V
- Precision (at each grid point) 10^{-6} V

Use the results to make a contour plot of the potential (filled or not, it's your choice).

Submit the plot.

- (b) Repeat, with over-relaxation. Try with $\omega = 0.1$ and $\omega = 0.5$. What do you notice?

Submit the plot and written answer.

- (c) Make a stream plot of the electric field lines, colour-coded by the value of the electric potential.

Submit the plot.

2. Shallow water system

- (a) Show that the 1D shallow water equations (8) can be rewritten in the flux-conservative form of Equation 3, with $\vec{u} = (u, \eta)$ and

$$\vec{F}(u, \eta) = \left[\frac{1}{2}u^2 + g\eta, (\eta - \eta_b)u \right]. \quad (9)$$

Then use the FTCS scheme (Equation 5) to discretize the 1D shallow water equations. You should have two equations of the form $u_j^{n+1} = \dots$ and $\eta_j^{n+1} = \dots$ where the right-hand sides depend on u and η at the timestep n . Assume $\eta_b(x)$ is a known function (i.e., $\eta_b(x_j) = \eta_{b,j}$ is given).

Submit written answer.

- (b) Implement the 1D shallow water system with the FTCS scheme in Python. Use the following set up (it's not particularly realistic – simulating waves in a very shallow 1D flat bathtub wouldn't be very useful – but we're trying to keep things simple in this lab):

- Domain $x = [0, L]$, with $L = 1$ m
- Grid spacing as shown in fig.1, with $J = 50$, so $\Delta x = 0.02$ m.
- Take $g = 9.81 \text{ m s}^{-2}$
- Flat bottom topography described by $\eta_b = 0$, $H = 0.01$ m
- Boundary conditions $u(0, t) = u(L, t) = 0$ (i.e. rigid walls)
- Timestep $\Delta t = 0.01$ s.
- Initial conditions:

$$u(x, 0) = 0, \quad \eta(x, 0) = H + Ae^{-(x-\mu)^2/\sigma^2} - \left\langle Ae^{-(x-\mu)^2/\sigma^2} \right\rangle, \quad (10)$$

with $A = 0.002$ m, $\mu = 0.5$ m, $\sigma = 0.05$ m, and where $\langle \rangle$ is the x -average operator, ensuring that H retains its definition of the free surface altitude at rest. This represents a small Gaussian peak at the centre of the domain.

Make plots of η vs x at $t = 0$ s, $t = 1$ s and $t = 4$ s.

Submit the code and plots.

Hints:

- You will have to treat the grid points at the boundaries separately, because you cannot use a centred difference approximation here, since you do not have values for u or η at $j = -1$ or $j = J + 1$. To get around this issue, use forward and backward differences at each of the boundary points to approximate the spatial derivative of the fluxes:

$$\left. \frac{\partial F}{\partial x} \right|_0^n \approx \frac{1}{\Delta x} (F_1^n - F_0^n), \quad \left. \frac{\partial F}{\partial x} \right|_J^n \approx \frac{1}{\Delta x} (F_J^n - F_{J-1}^n). \quad (11)$$

- You should find that your solution eventually diverges. Do not be alarmed, this doesn't necessarily mean you have an error in your code!
- In the finite difference scheme, you need to use *only* the previous time step values to update the variable. The easiest way to do this is by defining second arrays for u and η (e.g., called `u_new` and `eta_new`), and then your update lines in the loop can look like:

```
u_new[j] = u[j] + ... # (all in terms of u and eta)
eta_new[j] = eta[j] + ... # (all in terms of u and eta)
eta = np.copy(eta_new)
u = np.copy(u_new)
```

It is important to use the `np.copy` command in the last 2 lines, otherwise python just updates `eta` at the same time as `eta_new` in the next iteration. (Technically, this is because Python treats the equality sign as assignment to a pointer when the object on the right-hand side is a list or Numpy array. In other words, it's a deep copy, not a shallow copy. In other other words, it's a pass-by-reference, not a pass-by-value. If you don't know what any of these mean, don't worry.)

3. Waves with Burger's Equation

The FTCS method is numerically unstable when integrating the wave equation, of which the Burger's equation is an extension. Therefore we will instead use a leapfrog-based method to solve Equation (7):

$$\frac{u_i^{j+1} - u_i^{j-1}}{2\Delta t} = -\frac{\epsilon}{2} \frac{\left(u_{i+1}^j\right)^2 - \left(u_{i-1}^j\right)^2}{2\Delta x}, \quad (12)$$

or equivalently

$$u_i^{j+1} = u_i^{j-1} - \frac{\beta}{2} \left[\left(u_{i+1}^j \right)^2 - \left(u_{i-1}^j \right)^2 \right], \quad \text{with} \quad \beta = \epsilon \frac{\Delta t}{\Delta x}. \quad (13)$$

Note that you always need “starter” values at the edges of your space/time domain, for which the above equation does not apply. In space, this is taken care of by the boundary values, while in time, you need to initiate the leapfrog stepping with one forward Euler step.

(a) Implement this in Python, where your code should have parameters for:

- ϵ in Burgers Equation
- Grid spacing Δx
- Time step Δt
- Length of the spatial domain L_x
- Time duration for which the simulation runs, T_f .

The code should assume there is (already defined elsewhere) some function defining the initial condition $u(x, t = 0)$ and some function defining the boundary conditions $u(0, t)$, $u(L_x, t)$. The code should keep track of u at each grid point at each time step.

Submit the code.

- (b) Run your code with the following parameter values: $\epsilon = 1$, $\Delta x = 0.02$ m, $\Delta t = 0.005$ s, $L_x = 2\pi$ m, $T_f = 2$ s. Set the initial condition to be $u(x, t = 0) = \sin(x)$, and the boundary conditions to be $u(0, t) = 0$ and $u(L_x, t) = 0$.

Create plots of u vs x at each of the following times: 0, 0.5, 1, and 1.5 s. What is happening to the disturbance as time progresses? Does the solution seem realistic?

Submit the code, plots, and written answers.